

Appunti di Algorithm Engineering

Anno Accademico 2026/2027

Indice

1	Progettazione di Algoritmi, Modelli Computazionali e Gerarchia di Memoria	1
1.1	Definizione Formale di un Algoritmo	1
1.1.1	Algoritmi Esatti, Intelligenza Artificiale ed Euristiche	2
1.2	Misurazione dell'Efficienza e Analisi Asintotica	3
1.2.1	Perché Non Contare il Numero Esatto di Istruzioni?	3
1.2.2	Perché Preferire l'Analisi al Caso Pessimo?	4
1.3	Modelli Computazionali: RAM e Hardware Reale	4
1.3.1	Il Modello Random Access Machine	4
1.3.2	Hardware Reale: Gerarchia di Memoria e Memory Wall	5
1.3.3	Quando il Modello RAM Non È Sufficiente	5
1.4	Località e Modello a Due Livelli	7
1.4.1	Il Modello di Memoria a Due Livelli	7
1.5	Esempio Pratico: Scansione di un Array Lineare	9
1.5.1	Analisi nel Classico Modello RAM	9
1.5.2	Analisi nel Modello a Due Livelli	9
1.5.3	Accesso Lineare contro Accesso Sparso	10
2	Accesso a Passi, B-Tree e Impatto della Paginazione Virtuale	11
2.1	Accesso Non Sequenziale: L'Algoritmo $A_{s,b}$	11
2.1.1	Condizione di Coprimalità	12
2.1.2	Costo di I/O	12
2.2	Limiti Algoritmici e Accelerazione Hardware	12
2.3	Ricerca in Insiemi Ordinati: Ricerca Binaria contro B-Tree	13
2.3.1	Ricerca Binaria in Memoria Secondaria	13
2.3.2	B-Tree e B ⁺ -Tree	15
2.3.3	Confronto Quantitativo	16
2.4	Algoritmi I/O-Aware e Paginazione Virtuale	16
2.4.1	Paginazione con un Working Set Sovradimensionato	17
2.4.2	Esempio Numerico di Thrashing	17
2.5	Punti Chiave	18

3	Permutazione, Ordinamento e Merge Sort in Memoria Ester-	na	19
3.1	Il Problema della Permutazione vs. Ordinamento		19
3.1.1	Formulazione del Problema		19
3.1.2	Implementazione Ingenua su Modello RAM		20
3.1.3	Principio di Simulazione (RAM contro Due Livelli)		21
3.1.4	La Relazione tra Ordinamento e Permutazione		21
3.2	Complessità di Permutazione in Memoria Esterna		23
3.2.1	Analisi dei Casi Limite		25
3.2.2	Verifica Numerica su Parametri Architettureali Reali		25
3.3	Binary Merge Sort in Memoria Esterna		26
3.3.1	Architettura dei Buffer nella Procedura di Fusione		26
3.3.2	Complessità dell'Albero di Ricorsione		27
3.3.3	Criticità e Limiti di Efficienza		28

Capitolo 1

Progettazione di Algoritmi, Modelli Computazionali e Gerarchia di Memoria

Lezione 1

Introduzione, modelli computazionali e gerarchia di memoria

Data: 15 settembre 2026

1.1 Definizione Formale di un Algoritmo

Un *algoritmo* è una sequenza finita e ordinata di istruzioni elementari non ambigue che, a partire da un insieme specificato di dati di input, produce un corrispondente insieme di dati di output in un lasso di tempo finito.

Nella formulazione classica presentata in *The Art of Computer Programming* [1], un algoritmo deve soddisfare cinque proprietà fondamentali:

Finitudine. L'algoritmo deve sempre terminare dopo un numero finito di passi di calcolo per ogni configurazione di input ammissibile.

Definitezza. Ogni istruzione deve essere formulata in modo chiaro, preciso e senza ambiguità; lo stato di esecuzione successivo deve essere univocamente determinato.

Input. L'algoritmo riceve zero o più quantità specificate dall'esterno prima che l'esecuzione abbia inizio.

Output. L'algoritmo produce una o più quantità di output che hanno una relazione specificata con i dati di input. L'output deve essere significativo per il problema che si sta risolvendo.

Efficacia. Ogni operazione prescritta deve essere elementare e fisicamente eseguibile in tempo finito utilizzando una quantità limitata di risorse, come carta e penna o una macchina sequenziale.

1.1.1 Algoritmi Esatti, Intelligenza Artificiale ed Euristiche

Un classico algoritmo esatto si distingue da un metodo di intelligenza artificiale o euristico per le garanzie che fornisce e per il modo in cui esplora lo spazio delle decisioni.

Tabella 1.1: Confronto tra algoritmi esatti, intelligenza artificiale e metodi euristici.

Dimensione	Algoritmo esatto classico	Metodo di IA o euristico
Garanzia di correttezza	Produce una soluzione corretta per ogni istanza ammissibile; l'ottimalità è garantita quando la specifica del problema richiede un ottimo.	Può privilegiare la qualità empirica o l'efficienza pratica e può mancare di una garanzia universale per ogni istanza.
Terminazione	Può essere dimostrata in anticipo utilizzando funzioni potenziale o varianti di ciclo.	Spesso legata a criteri di arresto euristici, come un numero massimo di epoche o la convergenza del gradiente.
Spazio delle decisioni	Determinato da una procedura specificata in modo preciso, che può essere deterministica o randomizzata.	Esplorato tramite ricerca euristica, inferenza statistica o ottimizzazione di una funzione di perdita.

La *programmazione euristica* non cerca una dimostrazione formale di ottimalità assoluta su ogni istanza ammissibile. Piuttosto, cerca un compromesso efficiente per problemi computazionalmente intrattabili, come i problemi NP-completi o NP-ardui.

1.2 Misurazione dell'Efficienza e Analisi Asintotica

1.2.1 Perché Non Contare il Numero Esatto di Istruzioni?

Definire il costo esatto di esecuzione $T_A(n)$ come il numero effettivo di cicli di clock o di istruzioni elementari presenta diverse difficoltà:

1. **Dipendenza dall'architettura:** il risultato dipenderebbe dalla specifica CPU, dall'architettura del set di istruzioni (ISA) e dalla frequenza di clock.
2. **Dipendenza dalla toolchain:** varierebbe a seconda del compilatore, del livello di ottimizzazione e del sistema operativo.
3. **Mancanza di generalità:** oscurerebbe la qualità intrinseca dell'idea algoritmica.

Per questo motivo, l'analisi degli algoritmi utilizza la notazione asintotica. Essa astrae dai dettagli implementativi e dalle costanti moltiplicative, e studia il tasso di crescita della funzione di costo al tendere della dimensione dell'input all'infinito: $n \rightarrow \infty$.

O-grande (limite superiore asintotico):

$$f(n) = \mathcal{O}(g(n)) \iff \exists c > 0, n_0 > 0 \text{ tali che} \\ 0 \leq f(n) \leq c g(n), \quad \forall n \geq n_0.$$

Omega-grande (limite inferiore asintotico):

$$f(n) = \Omega(g(n)) \iff \exists c > 0, n_0 > 0 \text{ tali che} \\ 0 \leq c g(n) \leq f(n), \quad \forall n \geq n_0.$$

Theta-grande (limite asintotico stretto):

$$f(n) = \Theta(g(n)) \iff f(n) = \mathcal{O}(g(n)) \wedge f(n) = \Omega(g(n)).$$

1.2.2 Perché Preferire l'Analisi al Caso Pessimo?

- **Garanzia assoluta:** un limite superiore garantisce che l'algoritmo non impiegherà mai più tempo del limite teorico. Questo è essenziale per i sistemi real-time e mission-critical.
- **Semplice derivazione formale:** l'analisi al caso pessimo non richiede assunzioni probabilistiche sulla distribuzione dell'input, le quali sono spesso necessarie per l'analisi al caso medio.

1.3 Modelli Computazionali: RAM e Hardware Reale

Un algoritmo non viene eseguito in un vuoto astratto: viene eseguito su una macchina con processori, memorie e meccanismi di trasferimento dati. Un modello computazionale specifica quali dettagli della macchina sono rilevanti per l'analisi e quali vengono deliberatamente ignorati. Il modello RAM è il punto di partenza standard; l'hardware reale spiega poi quando tale astrazione diviene insufficiente.

1.3.1 Il Modello Random Access Machine

Il modello Random Access Machine (RAM) è un'astrazione sequenziale, basata su parole (word), ispirata all'architettura di von Neumann. Una RAM è costituita da un processore, un programma finito e una memoria indirizzabile di parole. Le sue assunzioni abituali sono:

- **Esecuzione sequenziale:** un processore esegue un'istruzione elementare alla volta.
- **Dati della dimensione di una parola:** l'aritmetica, i confronti, le assegnazioni e il calcolo degli indirizzi su un numero costante di parole richiedono tempo costante.
- **Accesso uniforme alla memoria:** la lettura o la scrittura di qualsiasi indirizzo costa $\mathcal{O}(1)$, indipendentemente dall'indirizzo e dalla storia degli accessi.
- **Memoria sufficiente:** l'input e le strutture dati risiedono interamente nello spazio di indirizzamento modellato; il costo di allocazione dello spazio viene analizzato separatamente quando risulta rilevante.

Il modello RAM è utile perché rende confrontabili le idee algoritmiche. Ad esempio, un ciclo che esamina n elementi di un array esegue $\Theta(n)$ operazioni RAM, indipendentemente dal particolare processore utilizzato per eseguirlo. Il modello non afferma che una macchina reale possieda memoria infinita o una latenza veramente costante; esso dichiara quali costi vengono astratti.

1.3.2 Hardware Reale: Gerarchia di Memoria e Memory Wall

Su una macchina moderna, il processore è molto più veloce delle memorie che contengono grandi insiemi di dati. Questo crescente divario tra calcolo e movimento dei dati è noto come *memory wall*. Inoltre, la memoria locale non è una singola risorsa uniforme: è organizzata in livelli con capacità, latenze e granularità di trasferimento differenti.

Una gerarchia semplificata, ordinata dal livello più veloce e più piccolo a quello più lento e più grande, è:

Registri della CPU $\longrightarrow$ Cache L1 $\longrightarrow$ Cache L2 $\longrightarrow$ Cache L3 $\longrightarrow$ DRAM
 $\longrightarrow$ NVMe/SSD/disco $\longrightarrow$ Storage remoto / cloud.

I registri e le cache fanno parte del sistema di memoria veloce del processore; la DRAM è l'usuale memoria principale; gli SSD e i dischi forniscono archiviazione persistente. Lo storage remoto e i servizi cloud sono inclusi qui solo come una più ampia gerarchia di accesso ai dati, in quanto sono raggiunti attraverso una rete piuttosto che attraverso il bus di memoria locale.

Figura 1.1 descrive i principali livelli di memoria locale: ogni strato inferiore scambia una minore velocità di accesso per una maggiore capacità.

Spostandosi verso il basso nella Figura 1.1, la capacità generalmente cresce mentre crescono anche la latenza e il costo di trasferimento. I rapporti esatti dipendono dall'hardware e dal carico di lavoro, per cui valori come da 100 a 200 volte per la DRAM rispetto alla L1, o da 10^4 a 10^6 volte per il disco rispetto alla L1, dovrebbero essere letti come esempi di ordine di grandezza piuttosto che come costanti universali.

1.3.3 Quando il Modello RAM Non È Sufficiente

Il modello RAM è una buona prima approssimazione quando il calcolo è prevalentemente sequenziale, il working set risiede interamente nella memoria principale e il costo del movimento dei dati non è il fattore dominante. Un modello diverso è preferibile quando uno dei seguenti effetti controlla le prestazioni:

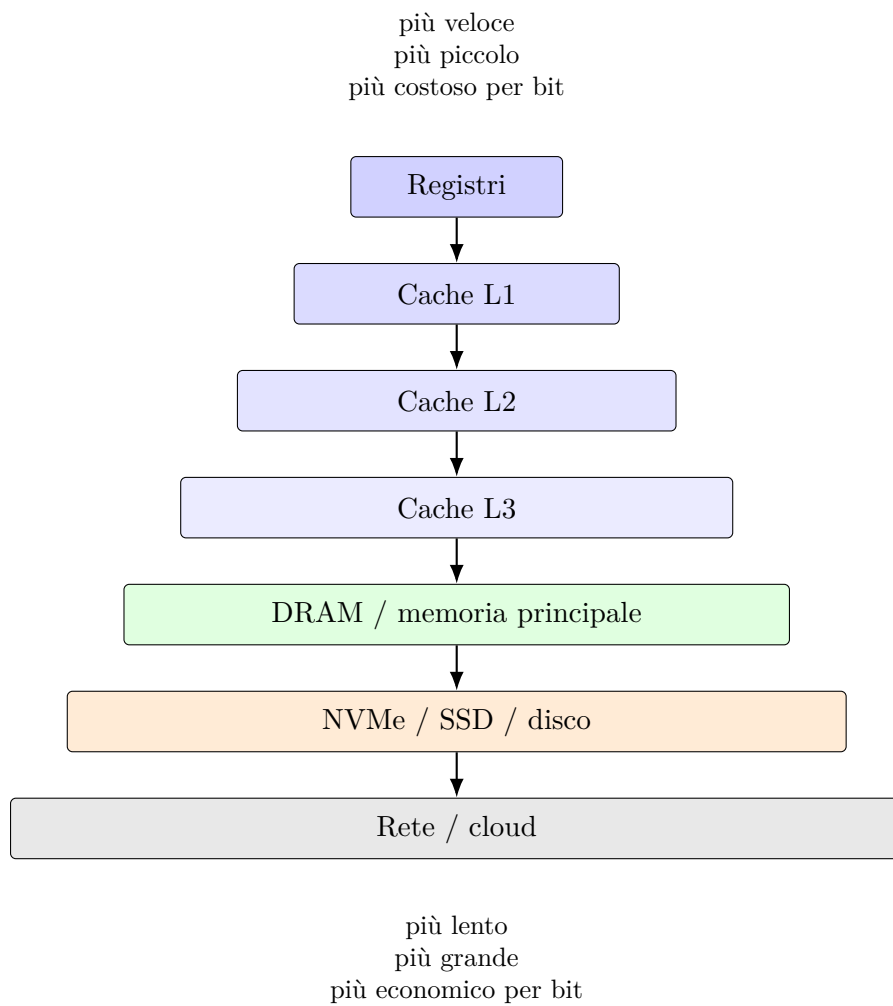


Figura 1.1: Una vista gerarchica della memoria e dell'archiviazione.

Calcolo su GPU. L'esecuzione SIMT (Single Instruction, Multiple Threads) utilizza molti core di elaborazione e distingue tra spazi di memoria condivisa, locale e globale.

Memoria esterna. Nel calcolo out-of-core, il dataset non risiede interamente nella memoria veloce, per cui i trasferimenti a blocchi e le operazioni di I/O dominano il costo.

Calcolo parallelo e distribuito. Diversi processori o macchine eseguono simultaneamente e la sincronizzazione e la comunicazione diventano costi espliciti.

Informatica quantistica. Qubit, sovrapposizione, entanglement e porte unitarie richiedono un modello basato su operazioni quantistiche piuttosto che su parole RAM.

La lezione ingegneristica è quindi non di scartare il modello RAM, ma di utilizzare il modello più semplice che catturi ancora il collo di bottiglia del sistema target.

1.4 Località e Modello a Due Livelli

I sistemi moderni sfruttano due proprietà statistiche dei programmi reali:

1. **Località temporale:** se una locazione di memoria viene referenziata al tempo t , è probabile che venga referenziata nuovamente a breve, al tempo $t + \Delta t$.
2. **Località spaziale:** se una locazione di memoria x viene referenziata, locazioni vicine come $x + 1$, $x + 2$ e così via, verranno probabilmente referenziate a breve.

1.4.1 Il Modello di Memoria a Due Livelli

Quando si elaborano gigabyte o terabyte di dati, la gerarchia multilivello può essere astratta come un modello a due livelli, associato al modello a memoria esterna [2], noto anche come modello Disk Access Machine.

Figura 1.2 descrive l'astrazione a due livelli: la CPU opera su parole nella memoria veloce M , mentre i trasferimenti tra M e la memoria esterna D avvengono in blocchi di B parole.

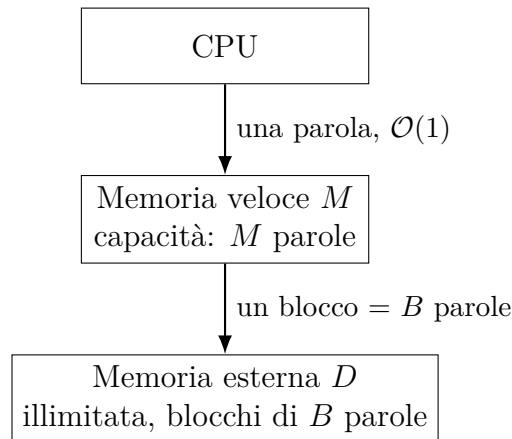


Figura 1.2: Modello di memoria a due livelli con memoria veloce M e memoria esterna D .

Parametri del Modello

N : numero totale di elementi del problema da elaborare.

M : dimensione della memoria veloce interna, misurata come il numero massimo di elementi che possono essere contenuti simultaneamente.

B : dimensione del blocco trasferito in una singola operazione di I/O, con $M \geq 2B$.

Metriche di Costo

1. **Complessità di I/O (metrica primaria):** numero di trasferimenti di blocchi di dimensione B tra la memoria lenta e quella veloce.
2. **Tempo di CPU (metrica secondaria):** numero di operazioni logiche e aritmetiche eseguite dopo che i dati sono stati caricati in M .
3. **Spazio:** numero di blocchi allocati sulla memoria secondaria.

Questa astrazione è frattale: può modellare sia i trasferimenti RAM–disco sia i trasferimenti cache L3–RAM, ridefinendo il significato di M e B .

1.5 Esempio Pratico: Scansione di un Array Lineare

Consideriamo un array contiguo di N interi scansionato in sequenza, ad esempio per calcolare la somma $\sum_{i=1}^N A[i]$.

Listing 1.1: Scansione sequenziale di un array

```
1 long sum = 0;
2 for (int i = 0; i < N; i++) {
3     sum += A[i];
4 }
```

1.5.1 Analisi nel Classico Modello RAM

- Accessi in memoria: N .
- Istruzioni di addizione: N .
- Tempo di calcolo asintotico: $\Theta(N)$.

1.5.2 Analisi nel Modello a Due Livelli

Assumiamo che la memoria secondaria memorizzi l'array A in modo consecutivo in blocchi di dimensione B e che $A[0]$ sia allineato con il confine di un blocco. Senza questa assunzione di allineamento, la scansione potrebbe richiedere al più un trasferimento di blocco aggiuntivo, senza alterare il limite asintotico:

1. Al primo accesso ad $A[0]$, si verifica un miss. Il sistema trasferisce il blocco contenente $A[0], \dots, A[B-1]$ nella memoria veloce con un singolo I/O.
2. I successivi $B-1$ accessi trovano i loro dati già nella memoria veloce, non richiedendo ulteriori I/O. Questo sfrutta la località spaziale.
3. Un nuovo trasferimento di I/O è necessario solo quando l'indice attraversa il confine di un blocco, ovvero, una volta ogni B elementi.

Pertanto, il costo totale di I/O di una scansione lineare è

$$\text{Scan}(N) = \left\lceil \frac{N}{B} \right\rceil = \Theta\left(\frac{N}{B}\right) \text{ operazioni di I/O.}$$

1.5.3 Accesso Lineare contro Accesso Sparso

- **Accesso sequenziale (alta località spaziale):** la lettura di N elementi richiede N/B operazioni di I/O. Poiché B varia tipicamente da 64 byte per una linea di cache a centinaia di kilobyte o megabyte per pagine e blocchi del disco, lo speedup rispetto al modello ingenuo può essere di un fattore approssimativamente pari a B .
- **Accesso casuale o passo $k > B$ (nessuna località spaziale):** ogni elemento richiesto potrebbe risiedere in un blocco diverso, causando N operazioni di I/O indipendenti. Il costo degrada a $\Theta(N)$ I/O, il che è asintoticamente più lento di un fattore B .

Capitolo 2

Accesso a Passi, B-Tree e Impatto della Paginazione Virtuale

Lezione 2

Accesso a passi, ricerca binaria contro B-Tree e paginazione virtuale

Data: 16 settembre 2026

Questo capitolo studia come il pattern di accesso di un algoritmo interagisca con la granularità di trasferimento di un sistema di memoria. Lo stesso lavoro aritmetico può avere costi molto differenti a seconda che gli accessi preservino la località spaziale, corrispondano alla dimensione del blocco, o superino la capacità della memoria veloce.

2.1 Accesso Non Sequenziale: L'Algoritmo

$A_{s,b}$

Sia S un array contenente n elementi. Il sistema di archiviazione trasferisce i dati in blocchi fisici di dimensione B (un blocco del disco o una linea di cache). Definiamo inoltre una dimensione del blocco logico b , dove $b \leq B$ e b divide B . I corrispondenti numeri di blocchi logici e fisici sono

$$N' = \frac{n}{b}, \quad N_{\text{phys}} = \frac{n}{B}.$$

L'algoritmo $A_{s,b}$ calcola la somma degli elementi visitando i blocchi logici con un pattern a passi (stride). Il blocco logico visitato al passo j è

$$\text{block}(j) = (js) \bmod N', \quad j = 0, 1, \dots, N' - 1.$$

2.1.1 Condizione di Coprimalità

La sequenza visita ogni blocco logico esattamente una volta se e solo se

$$\gcd\left(s, \frac{n}{b}\right) = 1.$$

In tal caso, la moltiplicazione per s è una permutazione dei residui modulo N' . Se il massimo comun divisore è $d > 1$, la sequenza si ripete dopo $N'/d = n/(bd)$ blocchi logici distinti e di conseguenza copre solo una frazione dell'array.

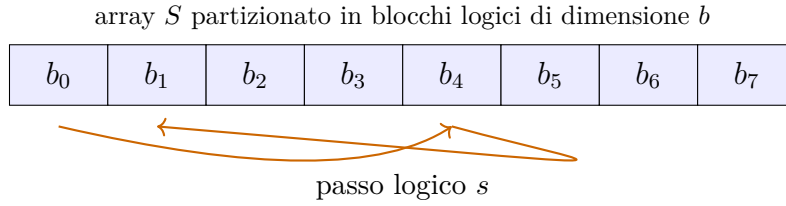


Figura 2.1: Un passo modulare visita i blocchi logici in un ordine non sequenziale.

La [Figura 2.1](#) illustra l'ordine logico degli accessi. Il costo fisico dipende da come questo ordine si mappa su blocchi di dimensione B , piuttosto che unicamente dal numero di operazioni aritmetiche.

2.1.2 Costo di I/O

Una prima utile approssimazione confronta la distanza fisica tra accessi logici successivi, sb , con la dimensione del blocco fisico B .

I limiti nella [Tabella 2.1](#) descrivono il tipico regime del caso pessimo o dei capacity miss. Le costanti esatte dipendono dall'allineamento, dalla politica di rimpiazzo e dall'interazione tra la sequenza modulare e i blocchi fisici. Nell'ultimo caso, il costo è peggiore di un fattore pari a circa B rispetto ad una scansione lineare, che può sfruttare ogni elemento trasferito.

2.2 Limiti Algoritmici e Accelerazione Hardware

Fissiamo un tempo di esecuzione massimo ammissibile T_0 e assumiamo che l'hardware o il parallelismo accelerino il calcolo di un fattore K . La macchina accelerata può pertanto permettersi un tempo di esecuzione originale pari a

$$\frac{T(n)}{K} = T_0 \quad \Longleftrightarrow \quad T(n) = KT_0.$$

Tabella 2.1: Comportamento approssimato di I/O del pattern di accesso a passi.

Condizione	Località e comportamento	Costo di I/O
$sb < B$	Diversi accessi logici possono condividere un singolo blocco fisico; una parziale località spaziale è preservata.	$\Theta(n/B)$
$sb \geq B$	Gli accessi successivi tipicamente ricadono su blocchi fisici differenti. Se $M < n$, il working set non può rimanere interamente nella memoria veloce.	$\Theta(n/b)$
$b = 1$ e $s \geq B$	Ogni accesso logico è un singolo elemento e il passo distrugge il beneficio dei trasferimenti a blocchi.	$\Theta(n)$

L'effetto sulla dimensione massima dell'input risolvibile dipende dal tasso di crescita di T .

La [Figura 2.2](#) mostra perché i miglioramenti hardware non rimuovono un algoritmo asintoticamente inefficiente. Per un algoritmo esponenziale, persino uno speedup di 1000 volte aumenta la dimensione dell'input risolvibile di soli $\log_2(1000) \approx 10$ elementi.

2.3 Ricerca in Insiemi Ordinati: Ricerca Binaria contro B-Tree

2.3.1 Ricerca Binaria in Memoria Secondaria

Per una sequenza ordinata di n chiavi memorizzate in blocchi di dimensione B , la ricerca binaria esegue $\Theta(\log_2 n)$ confronti sulla CPU. Il suo costo di I/O è maggiore di quanto il numero di confronti suggerisca: durante le prime iterazioni, le posizioni esaminate sono separate da $n/2$, $n/4$ e così via. Finché l'intervallo di ricerca non si restringe ad al più B elementi, ogni sondaggio

Tabella 2.2: Effetto di un'accelerazione hardware di un fattore K sulla dimensione dell'input risolvibile.

Algoritmo	Complessità	Hardware di base	Hardware accelerato	Guadagno effettivo
A_1	cn	$n_1 = T_0/c$	$n'_1 = Kn_1$	Moltiplicativo, K
A_2	cn^2	$n_2 = \sqrt{T_0/c}$	$n'_2 = \sqrt{K} n_2$	Sublineare, $\sqrt{K}$
A_3	$c2^n$	$n_3 = \log_2(T_0/c)$	$n'_3 = n_3 + \log_2 K$	Additivo, $+\log_2 K$

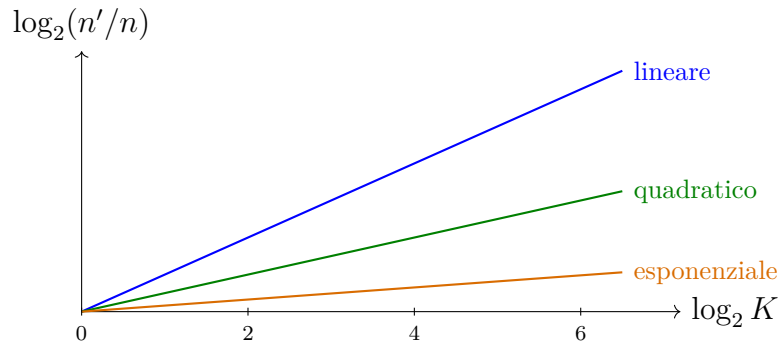


Figura 2.2: Beneficio relativo dell'accelerazione hardware per tassi di crescita rappresentativi.

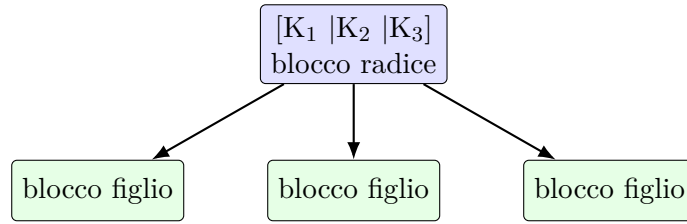
può indirizzare un blocco fisico differente. Pertanto,

$$I/O_{\text{ricerca binaria}} \approx \log_2 n - \log_2 B = \log_2 \left(\frac{n}{B} \right).$$

L'approssimazione ignora l'allineamento e la possibilità che un blocco rimanga nella cache, ma cattura il comportamento dominante in memoria esterna.

2.3.2 B-Tree e B⁺-Tree

Un B-Tree sceglie la dimensione del nodo in modo che corrisponda ad un blocco fisico. Ogni nodo memorizza $\Theta(B)$ chiavi e puntatori ai figli, pertanto il suo fan-out è $\Theta(B)$. Un B⁺-Tree usa lo stesso principio e normalmente memorizza i record o i puntatori ai record nelle sue foglie, rendendolo particolarmente adatto per le query ad intervallo.



ogni nodo occupa approssimativamente un blocco fisico B

Figura 2.3: Un B-Tree orientato ai blocchi con alto fan-out.

La [Figura 2.3](#) mostra la decisione fondamentale di layout: un I/O carica un intero nodo e fornisce molte chiavi di instradamento e puntatori. L'altezza, e di conseguenza il numero di letture di blocchi per una point query, è approssimativamente

$$h = \log_B \left(\frac{n}{B} \right) = \frac{\log_2(n/B)}{\log_2 B}.$$

Più precisamente, i dettagli implementativi introducono arrotondamenti per eccesso e fattori di occupazione. Lo spazio dei nodi interni è piccolo in confronto a quello delle foglie poiché

$$\text{nodi interni} \leq \frac{n}{B^2} + \frac{n}{B^3} + \dots = \frac{n}{B(B-1)} \ll \frac{n}{B}.$$

2.3.3 Confronto Quantitativo

Ignorando gli arrotondamenti per eccesso, lo speedup di I/O di un B-Tree rispetto alla ricerca binaria è

$$\frac{I/O_{\text{ricerca binaria}}}{I/O_{\text{B-Tree}}} = \frac{\log_2(n/B)}{\log_2(n/B)/\log_2 B} = \log_2 B.$$

Tabella 2.3: Confronto illustrativo per un grande dataset ordinato.

Parametro	Valore
Dimensione del dataset	$n = 2^{40}$ elementi, approssimativamente 10^{12} record
Dimensione del blocco	$B = 2^{14} = 16\,384$ record
I/O della ricerca binaria	$40 - 14 = 26$ accessi ai blocchi
I/O del B-Tree	$(40 - 14)/14 \approx 1.85$, quindi circa 2 accessi ai blocchi

Per una latenza di archiviazione su scala dei millisecondi, la differenza tra approssimativamente 26 e 2 accessi ai blocchi cambia una point query da centinaia di millisecondi a soli pochi millisecondi, subordinatamente alla cache e ai dettagli hardware.

2.4 Algoritmi I/O-Aware e Paginazione Virtuale

Ci sono due modi complementari per gestire i trasferimenti tra memoria veloce e lenta:

Progettazione I/O-aware (esplicita). L'algoritmo modella B e la capacità della memoria veloce M in modo diretto, e gestisce buffer, layout dei dati e trasferimenti in modo programmatico.

Memoria virtuale e paginazione (implicita). Il sistema operativo e l'hardware muovono le pagine in modo trasparente utilizzando tabelle delle pagine e politiche di rimpiazzo come le approssimazioni LRU.

2.4.1 Paginazione con un Working Set Sovradimensionato

Assumiamo che la memoria fisica abbia capacità M e sia divisa in M/B frame. Se l'applicazione richiede

$$n = (1 + \varepsilon)M = M + \varepsilon M, \quad \varepsilon > 0,$$

allora almeno εM unità del working set devono risiedere nella memoria secondaria. Sia α la frazione di istruzioni che eseguono operazioni di memoria. Un tipico profilo di istruzione è:

- frazione $1 - \alpha$: operazioni aritmetiche o logiche, con costo di circa un ciclo di CPU;
- frazione α : operazioni 'LOAD' o 'STORE';
- probabilità $1 - P(\varepsilon)$: i dati referenziati sono residenti in RAM, con tempo di accesso t_m ;
- probabilità $P(\varepsilon)$: l'accesso causa un page fault, con tempo di servizio t_{disco} .

Il tempo medio risultante per istruzione è

$$T_{\text{medio}} = (1 - \alpha) \cdot 1 + \alpha [(1 - P(\varepsilon))t_m + P(\varepsilon)t_{\text{disco}}].$$

Poiché $t_m \ll t_{\text{disco}}$, il termine del page fault può dominare:

$$T_{\text{medio}} \approx \alpha P(\varepsilon) t_{\text{disco}}.$$

Questa approssimazione è un avvertimento sulla sensibilità, non un'identità: quando $P(\varepsilon)$ è zero, i termini di base della CPU e della RAM hanno ancora rilevanza.

2.4.2 Esempio Numerico di Thrashing

Prendiamo $\alpha = 0.3$, una penalità per page fault di $t_{\text{disco}} = 10^5$ cicli di clock, e una probabilità di fault di soli $P(\varepsilon) = 10^{-3}$. Allora la penalità attesa è

$$T_{\text{penalità}} \approx 0.3 \cdot 10^{-3} \cdot 10^5 = 30 \text{ cicli per istruzione.}$$

Un page fault ogni mille accessi in memoria può pertanto aggiungere in media circa 30 cicli ad ogni istruzione. Rispetto ad una baseline di un ciclo, questo può produrre un rallentamento di un ordine di grandezza, potenzialmente nel range di $10\times$ – $30\times$ a seconda della latenza RAM ordinaria e dell'effettivo comportamento del working set. Quando il tasso di fault aumenta ulteriormente, il processo entra in *thrashing*: il sistema spende la maggior parte del suo tempo a servire page fault invece di progredire con computazione utile.

2.5 Punti Chiave

1. Un passo modulare visita tutti i blocchi logici solo quando il passo e il numero di blocchi logici sono coprimi.
2. Layout e pattern di accesso block-aware possono cambiare la complessità di I/O di un fattore B , anche quando il lavoro della CPU rimane inalterato.
3. I B-Tree riducono la profondità di ricerca in memoria esterna facendo corrispondere il loro fan-out alla dimensione del blocco fisico.
4. L'accelerazione hardware migliora gli algoritmi lineari e polinomiali in modi differenti, ma fornisce solo un miglioramento additivo agli algoritmi esponenziali.
5. La memoria virtuale è conveniente, ma un working set sovradimensionato può trasformare una piccola probabilità di page fault in un costo prestazionale dominante.

Capitolo 3

Permutazione, Ordinamento e Merge Sort in Memoria Esterna

Lezione 3

Permutazione vs Ordinamento e Merge Sort in Memoria Esterna

Data: 18 Settembre 2026

In questo capitolo viene esaminata la fondamentale relazione tra il problema della permutazione e quello dell'ordinamento nel modello di memoria a due livelli. A differenza del classico modello RAM, in cui permutare è un'operazione asintoticamente più economica rispetto all'ordinamento, nei sistemi a memoria gerarchica la gestione dei trasferimenti a blocchi introduce vincoli strutturali che legano intrinsecamente i due problemi. Viene inoltre introdotto e analizzato il Binary Merge Sort esterno, evidenziandone i limiti di efficienza e ponendo le basi per l'ordinamento multi-via.

3.1 Il Problema della Permutazione vs. Ordinamento

3.1.1 Formulazione del Problema

Siano dati due vettori di dimensione n :

- Un array di dati $S[1 \dots n]$.
- Un vettore di permutazione $\pi[1 \dots n]$, contenente una permutazione biiettiva degli indici $\{1, 2, \dots, n\}$.

L'obiettivo consiste nel riordinare gli elementi di S in un nuovo array S' in modo tale che ogni elemento originario $S[i]$ sia ricollocato nella posizione target indicata da $\pi[i]$, ovvero:

$$S'[\pi[i]] = S[i], \quad \forall i \in \{1, \dots, n\}. \quad (3.1)$$

Esempio 3.1 (Permutazione di indici). Consideriamo un array di 4 elementi e il relativo vettore di permutazione:

$$\begin{array}{lcl} \text{Indici:} & 1 & 2 & 3 & 4 \\ S = & [A, & B, & C, & D] \\ \pi = & [3, & 1, & 2, & 4] \end{array}$$

Applicando la definizione elemento per elemento:

$$\begin{aligned} S'[\pi[1]] &= S'[3] = A, \\ S'[\pi[2]] &= S'[1] = B, \\ S'[\pi[3]] &= S'[2] = C, \\ S'[\pi[4]] &= S'[4] = D. \end{aligned}$$

L'array permutato finale risulta pertanto:

$$S' = [B, \quad C, \quad A, \quad D].$$

In [Figura 3.1](#) viene illustrata visivamente la mappatura degli elementi dall'array originale al vettore permutato.

3.1.2 Implementazione Ingenua su Modello RAM

Nel modello RAM classico, in cui ogni accesso in lettura o scrittura a qualsiasi locazione di memoria ha costo unitario $\mathcal{O}(1)$, l'algoritmo si implementa tramite una singola scansione lineare:

Algorithm 1: Permutazione Diretta su RAM

Dati: Array $S[1 \dots n]$, vettore di permutazione $\pi[1 \dots n]$

Risultato: Array S permutato in loco

1 **for** $i \leftarrow 1$ **to** n **do**

2 $S'[\pi[i]] \leftarrow S[i]$

3 $S \leftarrow \text{copy}(S')$

La complessità temporale nel modello RAM è strettamente lineare:

$$T_{\text{RAM}}(n) = \Theta(n).$$

In memoria centrale, pertanto, permutare è asintoticamente più efficiente rispetto all'ordinamento basato su confronti, il quale richiede notoriamente un lower bound di $\Omega(n \log n)$ operazioni elementari.

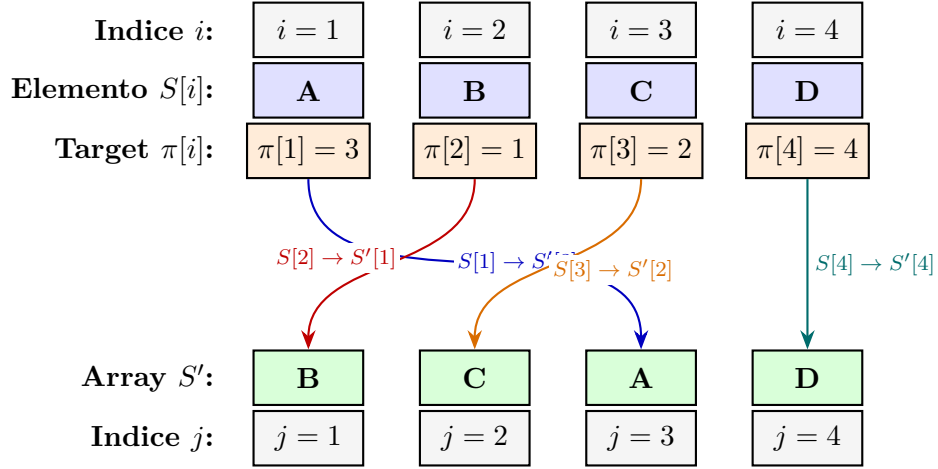


Figura 3.1: Mappatura della permutazione: ogni elemento $S[i]$ viene ricollocato nella posizione target specificata da $\pi[i]$ nell'array S' .

3.1.3 Principio di Simulazione (RAM contro Due Livelli)

Teorema 3.2 (Teorema di Simulazione). *Qualsiasi algoritmo sequenziale formulato per il modello RAM con complessità temporale $T(n)$ richiede al più $\mathcal{O}(T(n))$ trasferimenti di blocchi (I/O) nel modello di memoria a due livelli con parametri (M, B) .*

Dimostrazione (Idea). Nel caso peggiore, ogni istruzione elementare RAM che effettua un accesso in memoria (lettura o scrittura) indirizza una locazione appartenente a un blocco non presente in memoria veloce M . Ciascun accesso scatena pertanto un trasferimento di blocco dal disco, producendo esattamente 1 operazione di I/O. Moltiplicando il numero complessivo di passi $T(n)$ per il costo massimo di 1 I/O per passo, si ottiene il bound superiore immediato di $\mathcal{O}(T(n))$ I/O. $\square$

Sebbene il principio di simulazione garantisca che un algoritmo RAM possa sempre essere eseguito in memoria esterna, il bound $\mathcal{O}(T(n))$ è spesso inefficiente e lontano dall'ottimalità, poiché ignora completamente il fattore di aggregazione B offerto dai trasferimenti a blocchi.

3.1.4 La Relazione tra Ordinamento e Permutazione

Nel modello a due livelli la gerarchia di complessità si ribalta, come sintetizzato nel prospetto comparativo in [Tabella 3.1](#):

Problema	Modello RAM	Memoria a Due Livelli (I/O)
Permutazione Perm(n)	$\Theta(n)$	$\Theta\left(\min\left\{n, \text{Sort}(n, M, B)\right\}\right)$
Ordinamento Sort(n)	$\Theta(n \log_2 n)$	$\Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$

Tabella 3.1: Confronto di complessità asintotica tra RAM e Memoria Esterna.

Per comprendere questa equivalenza nei sistemi a blocchi, consideriamo le due riduzioni:

1. **Ordinamento tramite Permutazione:** Dato un array S , determinare la sequenza ordinata richiede dapprima di individuare la permutazione ordinante π_S (calcolabile tramite ordinamento delle chiavi) e successivamente di applicare π_S all'array S .
2. **Permutazione tramite Ordinamento (Algoritmo a Riduzione di Coppie):** La permutazione di un array S può essere trasformata in un problema di ordinamento di record secondo il seguente procedimento a tre stadi:

- **Passo 1 (Scansione e Creazione Coppie):** Si effettuano scansioni parallele e sequenziali di S e π per costruire una collezione di n record composti dalla coppia $\langle S[i], \pi[i] \rangle$. Poiché si tratta di una scansione sequenziale lineare, il costo è:

$$\text{I/O}_{\text{Passo 1}} = \mathcal{O}\left(\frac{n}{B}\right).$$

- **Passo 2 (Ordinamento per Destinazione):** Si ordinano le n coppie generate assumendo come chiave primaria la seconda componente $\pi[i]$. Il costo coincide esattamente con l'ordinamento in memoria esterna:

$$\text{I/O}_{\text{Passo 2}} = \text{Sort}(n, M, B).$$

- **Passo 3 (Scansione ed Estrazione):** Avendo ordinato le coppie per indice di destinazione crescente, è sufficiente una scansione lineare per estrarre la prima componente $S[i]$, riversandola nel vettore finale S' . Costo:

$$\text{I/O}_{\text{Passo 3}} = \mathcal{O}\left(\frac{n}{B}\right).$$

Il costo I/O aggregato di questa strategia è:

$$I/O_{\text{Sort-based}} = \mathcal{O}\left(\frac{n}{B}\right) + \text{Sort}(n, M, B) + \mathcal{O}\left(\frac{n}{B}\right) = \Theta(\text{Sort}(n, M, B)). \quad (3.2)$$

Esempio 3.3 (Tracciamento della Riduzione a Coppie con Array Reali). Per visualizzare concretamente la riduzione, consideriamo i dati dell'[Esempio 3.1](#):

$$S = [A, B, C, D], \quad \pi = [3, 1, 2, 4].$$

L'algoritmo opera attraverso tre stati successivi dell'array di record:

- **Passo 1 (Accoppiamento):** Si accoppia sequenzialmente ciascun elemento con la propria coordinata target:

$$P = [\langle A, 3 \rangle, \langle B, 1 \rangle, \langle C, 2 \rangle, \langle D, 4 \rangle].$$

- **Passo 2 (Sort esterno su $\pi[i]$):** L'array viene ordinato assumendo come chiave di comparazione il secondo valore della tupla (la destinazione finale):

$$P_{\text{ordinato}} = [\langle B, 1 \rangle, \langle C, 2 \rangle, \langle A, 3 \rangle, \langle D, 4 \rangle].$$

- **Passo 3 (Estrazione sequenziale):** Proiettando la prima coordinata di ciascun elemento ordinato, si ricompone il vettore finale:

$$S' = [B, C, A, D],$$

che realizza esattamente la permutazione originaria poiché $S'[1] = B$, $S'[2] = C$, $S'[3] = A$, $S'[4] = D$.

La pipeline di riduzione delle coppie è illustrata graficamente in [Figura 3.2](#).

3.2 Complessità di Permutazione in Memoria Esterna

Il lower bound fondamentale per il problema della permutazione in memoria a due livelli è stato stabilito da Aggarwal e Vitter [\[2\]](#):

$$\text{Perm}(n, M, B) = \Theta\left(\min\left\{n, \frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right)\right\}\right). \quad (3.3)$$

Definendo $L = \log_{M/B}(n/B)$ come il numero di passate richieste dalla fusione multi-via nell'ordinamento:

$$\text{Sort}(n, M, B) = \Theta\left(\frac{n}{B} \cdot L\right). \quad (3.4)$$

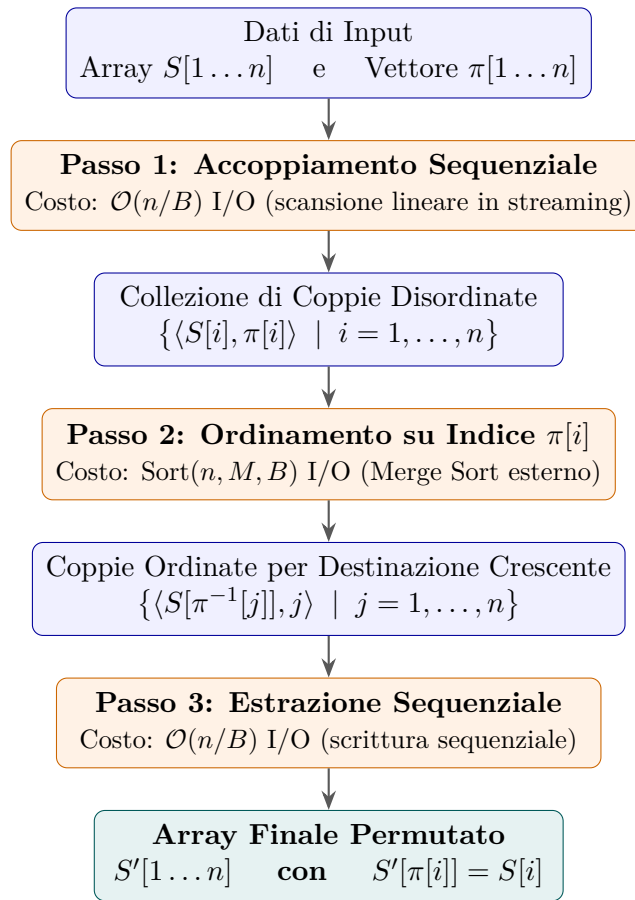


Figura 3.2: Flusso operativo dell'algoritmo di permutazione basato sulla riduzione a coppie.

3.2.1 Analisi dei Casi Limite

Il confronto tra l'algoritmo ingenuo diretto e la strategia basata sull'ordinamento delle coppie dipende dal valore relativo tra la dimensione del blocco B e il numero di livelli L :

- **Caso 1** ($n \geq \frac{n}{B} \cdot L \iff B \geq L$): La dimensione del blocco B è maggiore o uguale alla profondità dell'albero di fusione. In questa situazione, ordinare le coppie richiede meno I/O rispetto all'accesso casuale elemento per elemento:

$$\text{Perm}(n, M, B) = \Theta(\text{Sort}(n, M, B)).$$

- **Caso 2** ($n < \frac{n}{B} \cdot L \iff B < L$): Se la dimensione del blocco fosse patologicamente piccola rispetto al volume complessivo dei dati ($B < L$), eseguire n accessi diretti (al più n I/O) risulterebbe asintoticamente inferiore al costo dell'ordinamento:

$$\text{Perm}(n, M, B) = \Theta(n).$$

3.2.2 Verifica Numerica su Parametri Architettureali Reali

Per comprendere quale dei due regimi asintotici si manifesti nei sistemi reali, consideriamo parametri architetturali realistici ed estremi:

- Memoria interna veloce: $M = 16 \text{ GB} = 2^4 \cdot 2^{30} \text{ elementi} = 2^{34} \text{ elementi}$.
- Dimensione del blocco disco: $B = 32 \text{ KB} = 2^5 \cdot 2^{10} \text{ elementi} = 2^{15} \text{ elementi}$.
- Dimensione del dataset di input: $n = 2^{60} \text{ elementi}$ (scala Exabyte).

Calcoliamo il numero di passate L :

$$L = \log_{M/B} \left(\frac{n}{M} \right) = \frac{\log_2(n/M)}{\log_2(M/B)}.$$

Sostituendo i valori esponenziali:

$$\begin{aligned} \frac{n}{M} &= \frac{2^{60}}{2^{34}} = 2^{26} \implies \log_2 \left(\frac{n}{M} \right) = 26, \\ \frac{M}{B} &= \frac{2^{34}}{2^{15}} = 2^{19} \implies \log_2 \left(\frac{M}{B} \right) = 19. \end{aligned}$$

Otteniamo:

$$L = \frac{26}{19} \approx 1,36 < 2.$$

Poiché $B = 2^{15} = 32\,768$ e $L \leq 2$, la condizione $B \geq L$ è verificata con un margine di oltre quattro ordini di grandezza ($32\,768 \gg 2$). Di conseguenza, in qualsiasi sistema reale la permutazione è strettamente governata dal costo di ordinamento:

$$\text{Perm}(n) = \Theta(\text{Sort}(n)).$$

3.3 Binary Merge Sort in Memoria Esterna

L'algoritmo ricorsivo classico di Merge Sort si estende alla memoria esterna suddividendo l'array in due metà, ordinando ricorsivamente ciascuna metà e fondendo i due sottoarray ordinati (*runs*):

Algorithm 2: Binary Merge Sort

Dati: Array S , estremi i, j

Risultato: Sottoarray $S[i \dots j]$ ordinato

```

1 if  $i < j$  then
2    $m \leftarrow \lfloor (i + j)/2 \rfloor$ 
3    $\text{MS}(S, i, m)$ 
4    $\text{MS}(S, m + 1, j)$ 
5    $\text{MERGE}(S, i, m, j)$ 
```

3.3.1 Architettura dei Buffer nella Procedura di Fusione

Dati due segmenti ordinati (*runs*) consecutivi di lunghezza l , la procedura di fusione genera un unico run ordinato di lunghezza $2l$ su un array temporaneo S' .

- **Nel Modello RAM:** Si effettuano al più $2l$ confronti e $2l$ trasferimenti in memoria, determinando un costo computazionale di $\Theta(l)$.
- **Nel Modello a Due Livelli:** Per eseguire la fusione minimizzando gli I/O, la memoria interna M viene partizionata in 3 buffer dedicati, ciascuno di dimensione esattamente pari a un blocco B :

1. **Input Buffer 1:** ospita un blocco (dimensione B) del primo run ordinato.

2. **Input Buffer 2:** ospita un blocco (dimensione B) del secondo run ordinato.
3. **Output Buffer:** raccoglie gli elementi fusi in ordine crescente fino al riempimento di un blocco B .

In Figura 3.3 è rappresentata la gestione della memoria interna e dei flussi da/verso il disco durante la fusione binaria.

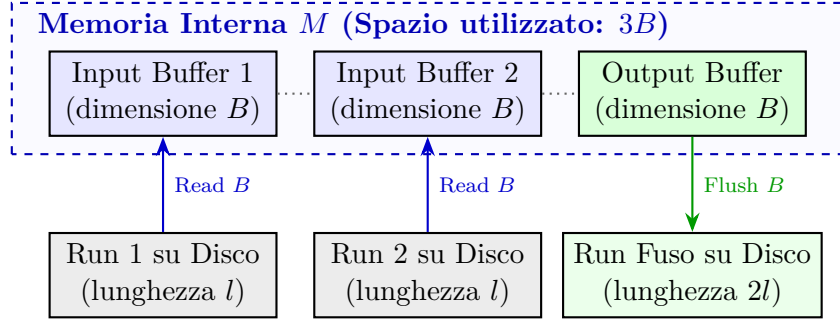


Figura 3.3: Architettura dei buffer di memoria interna durante la procedura di fusione binaria.

Il funzionamento del protocollo di trasferimento è il seguente:

1. Gli elementi in testa ai due buffer di input vengono confrontati in CPU; l'elemento minore viene spostato nell'output buffer.
2. Quando uno dei buffer di input si svuota, viene emessa una lettura da disco per caricare il blocco successivo del rispettivo run ($\frac{l}{B}$ I/O per ciascun run di input).
3. Non appena l'output buffer si riempie con B elementi, viene scaricato (*flushed*) su disco con una singola scrittura ($\frac{2l}{B}$ I/O complessivi di scrittura).

Il costo aggregato della fusione di due run di lunghezza l è:

$$\text{I/O}_{\text{Merge}}(l) = \frac{l}{B} + \frac{l}{B} + \frac{2l}{B} = \frac{4l}{B} = \Theta\left(\frac{l}{B}\right). \quad (3.5)$$

3.3.2 Complessità dell'Albero di Ricorsione

La ricorsione si arresta non appena la dimensione del sottoarray scende al di sotto della capacità della memoria interna M . A questo livello base, ciascun sottoarray di dimensione M viene letto interamente in memoria, ordinato

localmente tramite un algoritmo interno efficiente (es. QuickSort o HeapSort) con zero I/O aggiuntivi, e riscritto su disco con $\mathcal{O}(M/B)$ I/O.

Si generano così inizialmente:

$$N_{\text{run}} = \frac{n}{M} \text{ run ordinati di dimensione } M.$$

L'albero di fusione binaria presenta un'altezza complessiva pari a:

$$h = \log_2 \left(\frac{n}{M} \right) \text{ livelli.} \quad (3.6)$$

In Figura 3.4 è illustrata la struttura dell'albero di fusione per il Binary Merge Sort esterno.

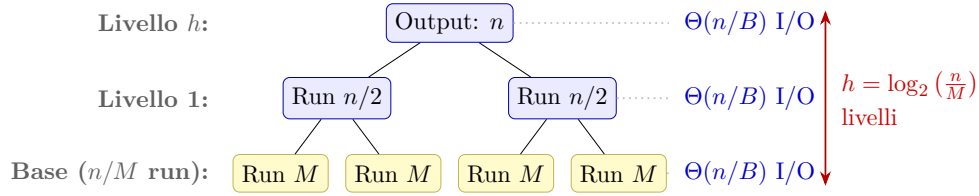


Figura 3.4: Albero di ricorsione del Binary Merge Sort esterno con indicazione dei costi I/O per livello.

Ad ogni livello dell'albero di fusione, tutti gli n elementi dell'array vengono scanditi esattamente una volta, richiedendo $\Theta(n/B)$ operazioni di I/O per livello. Moltiplicando per l'altezza dell'albero si ottiene la complessità totale:

$$\text{I/O}_{\text{BinaryMergeSort}} = \mathcal{O} \left(\frac{n}{B} \log_2 \left(\frac{n}{M} \right) \right). \quad (3.7)$$

3.3.3 Criticità e Limiti di Efficienza

Nonostante l'algoritmo migliori drasticamente rispetto a un approccio ingenuo, esso presenta un grave collo di bottiglia strutturale:

- **Sottoutilizzo drastico della memoria interna:** Durante la fusione binaria vengono occupati contemporaneamente soltanto 3 blocchi ($3B$ elementi) all'interno dello spazio disponibile M . Tutta la restante porzione di memoria interna ($M - 3B$) rimane completamente inutilizzata.
- **Base del logaritmo sub-ottimale:** La base del logaritmo è rigidamente ancorata a 2, mentre la memoria M permetterebbe di mantenere aperti simultaneamente fino a:

$$k \approx \frac{M}{B} - 1 \text{ buffer di input.}$$

- **Perdita del fattore di convergenza $\log_2(M/B)$:** La mancata estensione a una fusione a k vie (*Multiway Merge Sort*) comporta un numero di passate significativamente superiore, impedendo di raggiungere il lower bound ottimale:

$$\mathcal{O}\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right)\right).$$

In Figura 3.5 viene evidenziata graficamente la disparità tra l'allocazione binaria e la piena saturazione della memoria veloce.

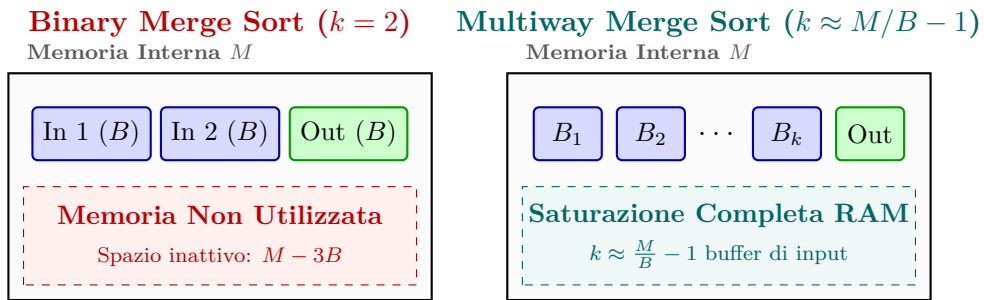


Figura 3.5: Confronto dell'efficienza nell'allocazione della memoria interna: grave sottoutilizzo nel Binary Merge Sort rispetto alla saturazione ottimale nel Multiway Merge Sort.

L'analisi del Multiway Merge Sort e il superamento di questa limitazione costituiranno l'oggetto delle lezioni successive.

Bibliografia

- [1] Donald E. Knuth. *The Art of Computer Programming*, volume 1. Addison-Wesley, Reading, MA, 3 edition, 1997.
- [2] Alok Aggarwal and Jeffrey S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.